

Protocol Analysis Report

Task 5 — Network Traffic Capture Using Wireshark

Date	June 7, 2026
Tool	Wireshark 4.6.6
Interface	Wi-Fi
Total Packets Captured	59,926 Dropped: 0 (0.0%)

1. TCP — Transmission Control Protocol

Filter used: tcp

Packets displayed: ~48,600 (81.2%)

Description:

TCP is a connection-oriented protocol that ensures reliable, ordered delivery of data between two devices. Before any data is sent, TCP performs a three-way handshake (SYN → SYN-ACK → ACK).

Observed in Capture:

- Continuous ACK, PSH, and FIN packets between local IP (192.168.254.228) and remote server (74.224.107.129)
- Port 80 (HTTP) and Port 443 (HTTPS) connections observed
- TCP retransmissions seen, indicating some packet loss and re-sending

Sample Packet Detail:

Source: 192.168.254.228 → Dest: 74.224.107.129 | Flags: [PSH, ACK] | Sequence numbers incrementing with each packet

2. DNS — Domain Name System

Filter used: dns

Packets displayed: 172 (0.3%)

Description:

DNS translates human-readable domain names (like google.com) into IP addresses. Every time you visit a website, a DNS query is sent first to resolve the domain. DNS uses UDP port 53.

Observed in Capture:

- Standard DNS queries (Type A and AAAA) from local machine to DNS server (192.168.254.138)
- Domains resolved: microsoft.com, cloudflare.com, digicert.com, assets.msn.com
- Both IPv4 (A record) and IPv6 (AAAA record) lookups observed

Sample Packet Detail:

Source: 192.168.254.228 → Dest: 192.168.254.138 | Query: fd-api.iris.microsoft.com | Response: CNAME and AAAA records

3. TLS — Transport Layer Security

Filter used: `tls`

Packets displayed: 1,205 (2.0%)

Description:

TLS is the security protocol that powers HTTPS. It encrypts data so that even if packets are intercepted, the contents cannot be read. TLS replaced the older SSL protocol.

Observed in Capture:

- TLS 1.2 and TLS 1.3 handshakes observed
- Client Hello packets — browser initiating a secure connection
- Server Hello — server responding with its certificate
- Application Data — encrypted web content being transferred
- SNI visible in Client Hello showing domains: api.iris.microsoft.com, assets.msn.com

Sample Packet Detail:

TLS 1.2 — Server Hello, Certificate, Server Key Exchange, Server Hello Done
| Port 443

4. HTTP — Hypertext Transfer Protocol

Filter used: `http`

Packets displayed: 88 (0.1%)

Description:

HTTP is the unencrypted version of web traffic. While most modern websites use HTTPS, HTTP traffic was observed mainly from background system services and streaming content requests.

Observed in Capture:

- GET requests to streaming service file paths observed
- HTTP 1.1 responses with 206 Partial Content (streaming/chunked data)

- HEAD requests to autoupdate.ini (Microsoft update checks)

Sample Packet Detail:

Method: GET | Response: HTTP/1.1 206 Partial Content | Content streamed in chunks from a media server

5. ICMPv6 — Internet Control Message Protocol v6

Filter used: icmpv6

Packets displayed: 73 (0.1%)

Description:

ICMPv6 is used for network diagnostics on IPv6 networks. The ping command uses ICMP to test connectivity — it sends an Echo Request and waits for an Echo Reply.

Observed in Capture:

- Echo (ping) request — generated by running ping google.com in Command Prompt
- Echo (ping) reply — response from Google's server (2404:6800:4007:83e::200e)
- Neighbor Solicitation/Advertisement — IPv6 equivalent of ARP
- Multicast Listener Report — device reporting multicast group memberships
- Router Advertisement — router announcing itself on the network

Sample Packet Detail:

Type: Echo (ping) request | id=0x0001 | seq=34 | hop limit=17 | Reply time: 90-180ms

Summary Table

Protocol	Packets	% of Total	Transport	Port
TCP	~48,600	81.2%	—	80, 443
DNS	172	0.3%	UDP	53
TLS	1,205	2.0%	TCP	443
HTTP	88	0.1%	TCP	80
ICMPv6	73	0.1%	—	—

Conclusion

The packet capture successfully demonstrated how multiple protocols work together during normal internet usage. TCP forms the backbone of most traffic, while DNS resolves domain names before connections are made. TLS secures HTTPS connections, making data unreadable to third parties. HTTP was observed for background service calls, and ICMPv6 enabled network diagnostics via the ping command. This hands-on capture builds a strong foundation for understanding how data actually travels across a network.